General questions

- Will the exam be hand-graded by you and the SLs? Are the function writing drill questions on the exam graded in a way that an error would result in the whole question being wrong or only some of the points being lost?
    - The exam is a mix of multiple choice and writing drills (48 questions - 45 of those are multiple choice (70% of the midterm grade) and 3 are function writing drills (30% of the midterm grade). The multiple choice questions are auto-graded and the writing drills hand-graded.
    - Function writing drills are all graded in a way that you can get partial credit if part of your answer is correct. They aren't “all or nothing”.
- What percent of our grade is the midterm?
    - 20%
- If we do better on the final, does that grade replace the midterm?
    - The syllabus states: “If you miss the midterm, your final will be worth 50% of your grade”. Although that isn’t the same as replacing your midterm grade if you do better on your final exam, I will allow students to have their final replace their midterm (ie drop the midterm and have the final be worth 50% of your final grade).
- How complex are the questions that are not multiple choice?
    - Each coding question is a self-contained problem to solve, similar *in complexity* to the worksheets, discussions, and programming assignments (They won’t be as long or as complex as something like play_turn(), though).
- Is the exam proctored by Examity and are we are allowed notes?
    - Although the syllabus states that the midterm will be proctored by Examity, I’ve decided against using it for the midterm (and possible the final) because I want students to be able to use their notes.
    - The exam is open note and you can use VS Code/Python shell to test your code*. That said, the exam is made so that your notes are more of a helpful reminder than something to rely on throughout. You won’t have time to consult and revise your notes for each question.

      * If you copy/paste your code from VS Code onto D2L, I highly recommend closing VS Code and starting from scratch immediately before copy/pasting to avoid any errors due to last minute changes or objects still in memory that you may have forgotten about.
- Can we use short hand or full type on the test? (e.g. point = point + 10 vs point += 10, or breaking down test scores versus using a while loop and if statements? Will the questions ask for one or the other?
    - The general rule of thumb is that, unless otherwise specified, as long as your code does *exactly* what we ask, it doesn’t matter how you get there. As such, using “point = point + 10” or “point += 10” shouldn’t matter. However, if the question specifically asks you to use a for loop you need to use a for loop. If we ask you to create a function that returns a value/variable, you need to define a function and that function needs to return that value/variable.

Content questions:

I'm a little bit confused with how you should end `while` loops versus `for` loops. For `while` loops, do you always use either '`break` ' or '`continue`' for ending it, or can you use '`return`'? I'm also just unsure when I should use '`return None`,' `return True`,' and '`return False`'?

> You can only use `return` inside of a function, and so you'll need to use `break` to break a loop outside of a function.
>
> We can use `return` to break a loop when we are inside a function (`return` will also end the function at the same point that the loop was interrupted) and we know that we want the function to return when a certain condition is met inside a loop.
>
> `continue` is used to skip the rest of the code (after the continue is reached) in an iteration and go straight to the beginning of the next iteration, we can't use `continue` to break out of a loop.
>
> `break` will break the loop when the command is reached, but only the loop in which it's found (i.e. a `break` inside of a nested loop will only break that loop and not all loops). It's the most straightforward way to break a loop once a condition has been met, but it may be an unnecessary step if we know we want to return a value from a function the loop is in (also, if you `break` in the line right above of your `return` your `return` will never be read by python). You can `return` inside the loop and, as the function ends, the loop will end by default. Break is the most straightforward way to do this, though!
>
> Generally, `while` loops vs `for` loops: the key difference is that with for loops, we generally know exactly how long (how many iterations) we want a loop to run and it will run for that specific number of iterations. While loops are useful when we don't know that information - when we don't have a set number of times we want a loop to iterate and/or when we want to use a condition to break the loop that is a non-numeric/length-based condition.
>
> `While True` loops are a special case of this where you want the loop to iterate indefinitely until a specific condition is met, at which point you'd call either a `break` or a `return` to end the loop (and function, if inside a function). Effectively, you could make a `While` loop with a condition instead of `While True` that achieves the same end, it's just a little bit more work than if you used a `While True` loop. *See example in the .py file on D2L*

What is the difference between / and //?

> / is a floating point division (i.e. regular division) – it may produce decimal output, if fractional output is needed.
>
> // integer division, outputs whole numbers (i.e. an even number of times that the number you're dividing by goes into the number being divided)
>
> 11 // 2 = 5, 11 % 2 = 1
>
> 11 / 2 = 5….

Can a parameter also be an argument? (In the All_about_functions pdf, they share some answers)

```
1  import math
2
3  def angle(x1, y1, z1, x2, y2, z2):
4      '''
5      Return the angle between the real-valued vectors
6      u = [x1, y1, z1] and v = [x2, y2, z2] in degrees.
7      '''
8      dot_product = x1 * x2 + y1 * y2 + z1 * z2
9      length_u = math.sqrt(x1**2 + y1**2 + z1**2)
10     length_v = math.sqrt(x2**2 + y2**2 + z2**2)
11     angle = math.acos(dot_product / length_u / length_v)
12     return angle * 180 / math.pi
13
14 print(angle(1, 0, 0, 0, 0, 1))
```

Great question! Yes. All_about_functions example

1. What line(s) contain function signatures?
   3
2. What line(s) contain function calls?
   9, 10, 11, 14
3. What line(s) contain variables?
   3, 8-12
4. List all of the variables.
   angle, math.pi, x1, x2, y1, y2, z1, z2, length_v, dot_product, length_u
5. What line(s) contain function bodies?
   4-12
6. What line(s) contain literals?
   9, 10, 12, 14
7. What line(s) contain parameters?
   3, 8-10
8. What line(s) contain arguments?
   9-11, 14
9. What line(s) contain assignment statements?
   8-11
10. Are function calls expressions?
    Yes - A function is an expression because a function evaluates something, that's all an expression is (don't worry about questions like this on the Midterm :-))
11. What line(s) contain return statements?
    12
12. How many function calls are there on line 14?
    2
13. What is the argument to the print function call on line 14?

The return value of angle given those arguments

Write a function called thirds that takes one string argument and returns a new string consisting of the third, sixth, ninth, etc. characters in the argument. If there aren't at least three characters, return None. What kind of traversal did you use?

```
def thirds(a_string):
    if len(a_string) < 3:
        return None
    new_string = ''
    i = 2 # the "third" element is at position 2, since indices start at 0.
    while i < len(a_string):
        new_string += a_string[i]
        i += 3 # increment by 3 since we only want every third element.
    return new_string
```

Can you talk about the continue and break in loops? Maybe a traceback with an example?

Example loop that does… something

```
added_variable = 0
for i in range(3):
    added_variable = added_variable + i
    for j in range(3):
        added_variable = added_variable + j
```

Iteration	added_variable	calculation	i	j
Before loops	0		-	-
1st i loop	0	(added_variable + i)	0	-
1st j loop	0	(added_variable + j)	0	0
2nd j loop	1	(added_variable + j)	0	1
3rd j loop	3	(added_variable + j)	0	2
2nd i loop	4	(added_variable + i)	1	-
1st j loop	4	(added_variable + j)	1	0
2nd j loop	5	(added_variable + j)	1	1
3rd j loop	7	(added_variable + j)	1	2
3rd i loop	9	(added_variable + i)	2	-
1st j loop	9	(added_variable + j)	2	0
2nd j loop	10	(added_variable + j)	2	1
3rd j loop	12	(added_variable + j)	2	2

What if we wanted to end the loops whenever `added_variable` is equal to 5?

What happens if we add a break in the j loop?

```
added_variable = 0
for i in range(3):
    added_variable = added_variable + i
    for j in range(3):
        added_variable = added_variable + j
        if added_variable == 5:
            break
```

Iteration	added_variable	calculation	i	j
Before loops	0		-	-
1st i loop	0	(added_variable + i)	0	-
1st j loop	0	(added_variable + j)	0	0
2nd j loop	1	(added_variable + j)	0	1
3rd j loop	3	(added_variable + j)	0	2
2nd i loop	4	(added_variable + i)	1	-
1st j loop	4	(added_variable + j)	1	0
2nd j loop	5	(added_variable + j)	1	1
3rd j loop			1	2
3rd i loop	7	(added_variable + i)	2	-
1st j loop	7	(added_variable + j)	2	0
2nd j loop	8	(added_variable + j)	2	1
3rd j loop	10	(added_variable + j)	2	2

```
            break
```

If we run this code, `added_variable` takes on the value of 10. But why?

j loop breaks here, skipping the 3rd iteration, but keeps going after that

Ok, so then let's put the break in the i loop then!

```
added_variable = 0
for i in range(3):
    added_variable = added_variable + i
    for j in range(3):
        added_variable = added_variable + j
        if added_variable == 5:
            break
    if added_variable == 5:
        break
```

If we run this code, `added_variable` takes on the value of 12. So, still not what we want, but why?

Iteration	added_variable	calculation	i	j
Before loops	0		-	-
1st i loop	0	(added_variable + i)	0	-
1st j loop	0	(added_variable + j)	0	0
2nd j loop	1	(added_variable + j)	0	1
3rd j loop	3	(added_variable + j)	0	2
2nd i loop	4	(added_variable + i)	1	-
1st j loop	4	(added_variable + j)	1	0
2nd j loop	5	(added_variable + j)	1	1
3rd j loop	7	(added_variable + j)	1	2
3rd i loop	9	(added_variable + i)	2	-
1st j loop	9	(added_variable + j)	2	0
2nd j loop	10	(added_variable + j)	2	1
3rd j loop	12	(added_variable + j)	2	2

The `if statement` is only read when in the I loop, and there's never a case where `added_variable` == 5 in the i loop, and so it will never break

And so, you'd need 2 `if statements`, one in each loop, to full close the loop and end while `added_variable` == 5

```
added_variable = 0
for i in range(3):
    added_variable = added_variable + i
    for j in range(3):
        added_variable = added_variable + j
        if added_variable == 5:
            break
    if added_variable == 5:
        break
```

Alternatively, if this loop is inside of a function, you could use `return` to close both loops (and the whole function) at once.

```
def add_stuff_and_stop_at_five(added_variable):
    added_variable = 0
    for i in range(3):
        added_variable = added_variable + i
        for j in range(3):
            if added_variable == 5:
                return added_variable
            added_variable = added_variable + j
```

You would have to move the `if statements` before the addition in the j loop though to make sure you're checking the value *after* adding in the i loop but *before* adding in the j loop